

Documento de prueba integral

Verificación del conversor

Curso 2025/2026



Realizado por Usuario de Prueba

Índice de contenidos

1. Introducción

- 1.1 Propósito
- 1.2 Primera tabla de la sección

2. Formato de texto

- 2.1 Énfasis
- 2.2 Párrafos y flujo
- 2.3 Hipervínculos

3. Listas

- 3.1 Lista no ordenada con guiones
- 3.2 Lista ordenada
- 3.3 Lista con texto largo

4. Bloques de código

- 4.1 Código sin lenguaje
- 4.2 Python con resaltado de sintaxis
- 4.3 Bash
- 4.4 JSON
- 4.5 Bloque de código largo (más de dos páginas)
- 4.6 Bloque mantenido junto al texto previo

5. Tablas

- 5.1 Tabla básica
- 5.2 Tabla con alineación

6. Imágenes

6.1 Primera imagen de la sección

6.2 Segunda imagen de la sección

7. Citas, separadores y elementos mixtos

7.1 Cita en bloque

7.2 Separador horizontal

7.3 Tabla e imagen en la misma sección

Índice de figuras

Figura 1.1: Montaña nevada al amanecer

Figura 1.2: Vista aérea de bosque

Figura 6.1: Perro de raza husky en la nieve

Figura 6.2: Montaña nevada al amanecer

Figura 7.1: Vista aérea de bosque

Índice de tablas

Tabla 1.1: Checklist de elementos verificables del PDF

Tabla 5.1: Parámetros de configuración del conversor

Tabla 5.2: Demostración de alineación de columnas

Tabla 7.1: Tabla de datos de ejemplo con tres columnas

Índice de bloques de código

Bloque de código 4.1: Ejemplo de uso en línea de comandos

Bloque de código 4.2: Función de numeración de figuras y tablas

Bloque de código 4.3: Script de instalación y primera conversión

Bloque de código 4.4: Parámetros de la llamada Page.printToPDF

Bloque de código 4.5: Módulo CsvAnalyser — estadísticas descriptivas completas

Bloque de código 4.6: Bloque forzado a seguir al párrafo anterior

1. Introducción

Este documento cubre todos los elementos que el conversor puede generar. Sirve como herramienta de verificación visual: cada sección comprueba un aspecto distinto del PDF resultante.

1.1 Propósito

El objetivo es confirmar que portada, índice, cabecera, pie, fuentes, saltos de página y numeración automática de figuras y tablas funcionan correctamente.

A continuación se muestra la primera imagen del documento:



Figura 1.1: Montaña nevada al amanecer

Y la segunda imagen de esta sección:



Figura 1.2: Vista aérea de bosque

1.2 Primera tabla de la sección

Elemento	Estado esperado	Notas
Portada	Título, autor, asignatura	Con logo si existe
Índice	Secciones y subsecciones	Con hipervínculos
Cabecera/pie	Título · Asignatura / Autor + pág.	Fuente embebida
Figuras	Figura x.y bajo la imagen	Contador por sección
Tablas	Tabla x.y encima	Contador por sección

Tabla 1.1: Checklist de elementos verificables del PDF

2. Formato de texto

El cuerpo admite el Markdown habitual.

2.1 Énfasis

- **Negrita** con doble asterisco.
- *Cursiva* con un asterisco.
- ***Negrita y cursiva*** combinadas.
- ~~Tachado~~ (no soportado por el conversor, se muestra en bruto).
- código en línea con acento grave.

2.2 Párrafos y flujo

Un párrafo normal tiene una separación cómoda con el siguiente. Este texto es suficientemente largo para comprobar que el interlineado de 1,6 y el tamaño de 13 pt resultan legibles en el PDF impreso.

Segundo párrafo de la misma subsección, sin ninguna marca especial.

2.3 Hipervínculos

Un enlace externo: [Página de inicio de Python](#). Los hipervínculos del índice y del outline del PDF son internos y se generan automáticamente; no es necesario escribirlos a mano.

3. Listas

3.1 Lista no ordenada con guiones

- Primer elemento de nivel 1
- Segundo elemento de nivel 1
- Subelemento anidado A
- Subelemento anidado B
 - Tercer nivel de anidamiento
- Tercer elemento de nivel 1

3.2 Lista ordenada

1. Paso uno
2. Paso dos
3. Paso tres
4. Subpaso 3.1
5. Subpaso 3.2
6. Paso cuatro

3.3 Lista con texto largo

- Este ítem tiene un texto deliberadamente largo para comprobar que el ajuste de línea funciona correctamente dentro del elemento de lista sin romper el sangrado ni el interlineado.
- Ítem corto.
- Otro ítem con **negrita** y código en línea mezclados con texto normal.

4. Bloques de código

4.1 Código sin lenguaje

```
$ md-to-pdf informe.md
informe.md → informe.pdf [OK, 142 KB]
```

Bloque de código 4.1: Ejemplo de uso en línea de comandos

4.2 Python con resaltado de sintaxis

```
def add_figure_table_numbers(html):
    section = [0]
    figs = [0]
    tabs = [0]
    pattern = re.compile(r'<h2\b[^>]*>|<img\b[^>]*/?>|
<table\b[^>]*>')

    def sub(m):
        tag = m.group(0)
        lo = tag.lower()
        if lo.startswith('<h2'):
            section[0] += 1
            figs[0] = 0
            tabs[0] = 0
            return tag
        if lo.startswith('<img'):
            figs[0] += 1
            return f'<figure>{tag}<figcaption>Figura {section
[0]}.{figs[0]}</figcaption></figure>'
        return tag

    return pattern.sub(sub, html)
```

Bloque de código 4.2: Función de numeración de figuras y tablas

4.3 Bash

```
# Instalar dependencias y lanzar la conversión
cd ~/.local/scripts/md-to-pdf
bash install.sh
md-to-pdf documento.md
```

Bloque de código 4.3: Script de instalación y primera conversión

4.4 JSON

```
{
  "id": 1,
  "method": "Page.printToPDF",
  "params": {
    "printBackground": true,
    "paperWidth": 8.27,
    "paperHeight": 11.69,
    "marginTop": 1.15
  }
}
```

Bloque de código 4.4: Parámetros de la llamada Page.printToPDF

4.5 Bloque de código largo (más de dos páginas)

El siguiente bloque verifica que el conversor maneja correctamente bloques de código que superan el límite de una página.

```

"""
Módulo de procesamiento de datos CSV con estadísticas
descriptivas.
Ejemplo extenso utilizado como caso de prueba para la
paginación de
bloques de código en el conversor md-to-pdf.
"""

from __future__ import annotations

import csv
import math
import statistics
from collections import defaultdict
from dataclasses import dataclass, field
from pathlib import Path
from typing import Any, Dict, Iterable, List, Optional, Tuple

@dataclass
class ColumnStats:
    name: str
    count: int = 0
    missing: int = 0
    numeric: bool = True
    values: List[float] = field(default_factory=list)
    categories: Dict[str, int] =
field(default_factory=lambda: defaultdict(int))

    # — Estadísticos numéricos

    @property
    def mean(self) -> Optional[float]:
        return statistics.mean(self.values) if self.values else None

    @property
    def median(self) -> Optional[float]:
        return statistics.median(self.values) if self.values
    else None

    @property
    def stdev(self) -> Optional[float]:
        return statistics.stdev(self.values) if len(self.values) > 1
    else None

```

```

@property
def minimum(self) -> Optional[float]:
    return min(self.values) if self.values else None

@property
def maximum(self) -> Optional[float]:
    return max(self.values) if self.values else None

@property
def q1(self) -> Optional[float]:
    if not self.values:
        return None
    s = sorted(self.values)
    return statistics.median(s[: len(s) // 2])

@property
def q3(self) -> Optional[float]:
    if not self.values:
        return None
    s = sorted(self.values)
    mid = (len(s) + 1) // 2
    return statistics.median(s[mid:])

@property
def iqr(self) -> Optional[float]:
    q1, q3 = self.q1, self.q3
    return q3 - q1 if q1 is not None and q3 is not None else None

def outliers(self, k: float = 1.5) -> List[float]:
    """Devuelve los valores fuera del rango [Q1 - k·IQR,
    Q3 + k·IQR]."""
    iqr = self.iqr
    if iqr is None:
        return []
    lo = self.q1 - k * iqr
    hi = self.q3 + k * iqr
    return [v for v in self.values if v < lo or v > hi]

def histogram(self, bins: int = 10) -> List[Tuple[float,
float, int]]:
    """Devuelve una lista de (límite_inf, límite_sup,
    frecuencia)."""
    if not self.values:
        return []
    lo, hi = self.minimum, self.maximum

```

```

    if lo == hi:
        return [(lo, hi, len(self.values))]
    width = (hi - lo) / bins
    counts = [0] * bins
    for v in self.values:
        idx = min(int((v - lo) / width), bins - 1)
        counts[idx] += 1
    return [(lo + i * width, lo + (i + 1) * width, c)
            for i, c in enumerate(counts)]

```

— Resumen

```

def summary(self) -> Dict[str, Any]:
    if self.numeric:
        return {
            "column": self.name,
            "type": "numeric",
            "count": self.count,
            "missing": self.missing,
            "mean": round(self.mean, 4) if self.mean is
s not None else None,
            "median": round(self.median, 4) if self.medi
an is not None else None,
            "stdev": round(self.stdev, 4) if self.stdev
is not None else None,
            "min": self.minimum,
            "q1": round(self.q1, 4) if self.q1 is no
t None else None,
            "q3": round(self.q3, 4) if self.q3 is no
t None else None,
            "max": self.maximum,
            "outliers": len(self.outliers()),
        }
    top = sorted(self.categories.items(), key=lambda x: -
x[1])[:5]
    return {
        "column": self.name,
        "type": "categorical",
        "count": self.count,
        "missing": self.missing,
        "unique": len(self.categories),
        "top_values": top,
    }

```

```
class CsvAnalyser:
```

"""Lee un CSV y calcula estadísticas descriptivas por columna."""

```
def __init__(self, path: str | Path, delimiter: str =
",", encoding: str = "utf-8"):
    self.path = Path(path)
    self.delimiter = delimiter
    self.encoding = encoding
    self.columns: Dict[str, ColumnStats] = {}
    self._row_count = 0
    self._loaded = False
```

— Carga

```
def load(self) -> "CsvAnalyser":
    with self.path.open(encoding=self.encoding, newline="
") as fh:
        reader = csv.DictReader(fh, delimiter=self.delimi
ter)

        if reader.fieldnames is None:
            raise ValueError("El archivo CSV no tiene
cabecera.")

        for name in reader.fieldnames:
            self.columns[name] = ColumnStats(name=name)
        for row in reader:
            self._row_count += 1
            for name, raw in row.items():
                col = self.columns[name]
                col.count += 1
                if raw is None or raw.strip() == "":
                    col.missing += 1
                    continue
                try:
                    col.values.append(float(raw))
                except ValueError:
                    col.numeric = False
                    col.categories[raw.strip()] += 1

            self._loaded = True
        return self
```

— Consultas

```
@property
def row_count(self) -> int:
```

```

        return self._row_count

    def numeric_columns(self) -> List[str]:
        return [n for n, c in self.columns.items() if c.numer
ic]

    def categorical_columns(self) -> List[str]:
        return [n for n, c in self.columns.items() if not c.n
umeric]

    def correlation(self, col_a: str, col_b: str) -> float:
        """Coeficiente de correlación de Pearson entre dos
columnas numéricas."""
        a = self.columns[col_a].values
        b = self.columns[col_b].values
        n = min(len(a), len(b))
        if n < 2:
            return float("nan")
        mean_a = sum(a[:n]) / n
        mean_b = sum(b[:n]) / n
        num = sum((a[i] - mean_a) * (b[i] - mean_b) for i in
range(n))
        den = math.sqrt(
            sum((a[i] - mean_a) ** 2 for i in range(n))
            * sum((b[i] - mean_b) ** 2 for i in range(n))
        )
        return num / den if den else float("nan")

    def report(self) -> List[Dict[str, Any]]:
        if not self._loaded:
            raise RuntimeError("Llama a load() antes de
report().")
        return [col.summary() for col in self.columns.values(
)]

    def __repr__(self) -> str:
        status = f"{self._row_count} rows" if self._loaded el
se "not loaded"
        return f"CsvAnalyser({self.path.name!r}, {status})"

```

Bloque de código 4.5: Módulo CsvAnalyser — estadísticas descriptivas completas

4.6 Bloque mantenido junto al texto previo

El marcador `<!-- keep -->` (en la línea anterior al elemento) fuerza a este a permanecer en la misma página que el contenido precedente, en lugar de empujarlo a la página siguiente. El siguiente bloque de código debe quedar pegado a este párrafo:

```
def keep_example():  
    """Este bloque se mantiene junto al texto que lo  
    introduce."""  
    return "permanece junto al párrafo previo"
```

Bloque de código 4.6: Bloque forzado a seguir al párrafo anterior

5. Tablas

5.1 Tabla básica

Nombre	Tipo	Valor por defecto
margin	float	1.15
font	string	Source Serif 4
size	int	13

Tabla 5.1: Parámetros de configuración del conversor

5.2 Tabla con alineación

Izquierda	Centro	Derecha
texto	texto	1 234,56 €
texto largo	texto largo	99,00 €
A	B	0,01 €

Tabla 5.2: Demostración de alineación de columnas

6. Imágenes

6.1 Primera imagen de la sección

Esta imagen recibe la etiqueta **Figura 6.1:**



Figura 6.1: Perro de raza husky en la nieve

6.2 Segunda imagen de la sección

Esta imagen recibe la etiqueta **Figura 6.2:**



Figura 6.2: Montaña nevada al amanecer

El contador se reinicia en la siguiente sección (##), por lo que la primera imagen de la sección 7 será *Figura 7.1*.

7. Citas, separadores y elementos mixtos

7.1 Cita en bloque

Esta es una cita en bloque (*blockquote*). Puede contener **texto con formato**, código e incluso varias líneas consecutivas que se unen en un único bloque visual con borde izquierdo y fondo gris claro.

Segunda cita independiente, separada de la anterior.

7.2 Separador horizontal

El siguiente elemento es una línea horizontal (`---` en el Markdown):

El texto continúa tras el separador. Observa que el conversor ignora el `---` inicial del documento (lo usa como separador portada/cuerpo), pero dentro del cuerpo produce el `<hr>` habitual.

7.3 Tabla e imagen en la misma sección

Esta sección tiene los dos tipos para verificar que los contadores son independientes:

Columna 1	Columna 2	Columna 3
A1	B1	C1
A2	B2	C2

Tabla 7.1: Tabla de datos de ejemplo con tres columnas



Figura 7.1: Vista aérea de bosque

Los elementos anteriores deben aparecer como **Tabla 7.1** y **Figura 7.1** respectivamente.